CSC 710 Software Engineering

Instructor: Dr. Tianxiao Zhang

My Research

- Computer Vision, Machine Learning, Deep Learning, Multimodal Learning

- Neural Networks, Transformers
- Image classification and recognition
- Object detection, tracking, segmentation
- Generative AI (e.g. GANs, Diffusion models)
- Multimodal Large Language Models, Vision-Language Models
- Transfer Learning, Domain Adaptation
- 3D Vision (e.g. 3D point cloud analysis, 3D detection)

- Applications: autonomous driving, face detection and recognition, video surveillance, agriculture, medical image analysis, etc

Office Hours and Email

- Office: IN-205
- Office hours

Wednesday 2:00 pm – 4:00 pm

Thursday 1:00 pm – 2:00 pm

By appointment

Email address: Tianxiao.Zhang@csi.cuny.edu

This Course

- Topics include scope of software engineering, life-cycle models, software process, teams, ethics, tools, testing, planning, and estimating.

- This course concentrates on requirements, analysis, design, implementation, and maintenance of software products.

- No textbook is required.

Course Outcomes

- Specifying, designing, analyzing, and documenting a significant software system
- Functioning effectively on teams
- Understanding professional, ethical legal, security, and social issues and responsibilities
- Communicating effectively with a range of audiences
- Analyzing the local and global impact of computing on individuals, organizations, and society
- Recognizing the need for, and an ability to engage in continuing professional development

Student Outcomes

- Analyze a complex computing problem and to apply principles of computing and other relevant disciplines to identify solutions
- Design, implement, and evaluate a computing-based solution to meet a given set of computing requirements in the context of the program's discipline
- Communicate effectively in a variety of professional contexts
- Recognize professional responsibilities and make informed judgments in computing practice based on legal and ethical principles
- Function effectively as a member or leader of a team engaged in activities appropriate to the program's discipline
- Apply computer science theory and software development fundamentals to produce computing-based solutions

Grading Policy

- A: 90 – 100
- A-: 85 – 89.9
- B+: 80 – 84.9
- B: 75 – 79.9
- B-: 70 – 74.9
- C+: 65 – 69.9
- C: 60 – 64.9
- F: below 60

Required Attendance

- The attendance will be recorded when the projects begin. You will need to report the project progress during the class time. If you do not attend the class without any reason, you will lose both attendance point and progress reporting point.

- You can email me before the lecture if you cannot attend the class and have an excused reason.

Grade Distributions

- Attendance - 10%
- Project 1 – 25 % (progress reporting 10%)
- Project 2 – 25 % (progress reporting 10%)
- Final Project (select your own topic) – 40% (progress reporting 10%)

- No midterm and final exams!
- Late submission will receive penalty. 1 day late will lose 10%, 2 days late will lose 20%, and so on so force until there is no point left.

Team-based Projects

- All projects are team based!
- 2 or 3 students in each team.
- Choose your own teammates.
- If the projects are assigned to you, each member of the team will report the progress to me for individual work evaluation during the class.

What is Software Engineering

- Software Engineering is the process of designing, developing, testing, and maintaining software.

- It is a systematic and disciplined approach to software development that aims to create high-quality, reliable, and maintainable software.

- It includes a variety of techniques, tools, and methodologies, including requirements analysis, design, testing, and maintenance.

Why is Software Engineering required?

- To manage Large software
- For more Scalability
- For Cost Management
- To manage the dynamic nature of software
- For better quality Management

Who is Software Engineer

- Software engineers are computer science professionals who use knowledge of engineering principles and programming languages to build software products, develop computer games, and run network control systems.

Types of Software Engineers

- The software engineering field is broad. Developers have different sets of technical expertise, from building computer information systems to maintaining network security to creating customer-facing web pages. There are two primary types of software engineers: applications software developers and systems software developers.

Types of Software Engineers

- **Applications Software Developers**

Design software for the end user to interact with them, Develop applications for iOS, Android, Windows, Linux, and more Conduct requirements analysis

- **Systems Software Developers**

Build operating systems and networks for user-facing applications. Responsible for both the hardware and software needs Integrate disparate software products onto one platform. Often serve as general IT managers or systems architects Design and enforce IT standards.

Team Organization

Most products are too large to be completed by a single software professional

- Software products must be assigned to a team
- Programming tasks require interaction among team members

Two extreme approaches to programming team organization:

- Democratic teams
- Chief programming teams

Democratic Team Approach

- The basic concept is egoless programming
- Programmers can be highly attached to their code – Programmers are not going to try to find faults in their own code
- Restructuring of social environment for programming
  - Programmers encouraged to find faults
  - A team will develop a group identity
- A group of up to 10 egoless programmers
  - No single leader
  - Management may have difficulty with such a team

Democratic Team Approach

- Advantage:

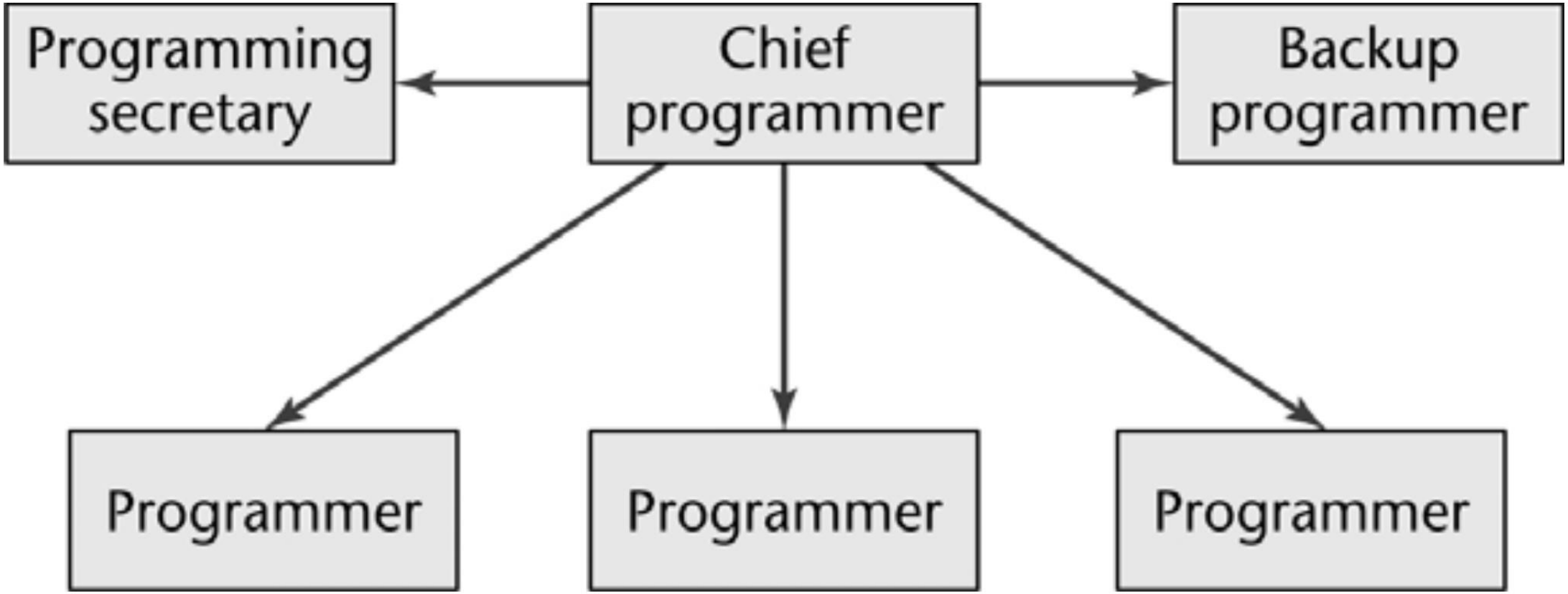
Positive attitude toward finding faults

- Disadvantage:

Senior programmers unhappy with beginners appraising their code

- Must spring up spontaneously and cannot be imposed from outside

Classical Chief Programmer Team Approach



Classical Chief Programmer Team Approach

- Two key aspects of chief programming team:

Specialization - Each member carries out only tasks for which trained

Hierarchy - A chief directing action of all the other members

- Reduction in the number of communication channels
- Chief programmer
  - Successful manager and highly skilled programmer
  - Works on the architectural design
  - Works on critical or complex sections of code

Classical Chief Programmer Team Approach

- Back-up programmer
  - In case the chief programmer needs to be replaced
  - Has to be as competent as the chief programmer
- Programming secretary
  - Highly skilled, central member of the team
  - Maintains project production library (documentation of project)
  - Source code listings, test data
  - Compiles, links, loads, executes, and runs tests programs
  - Some organizations use the term librarian

Classical Chief Programmer Team Approach

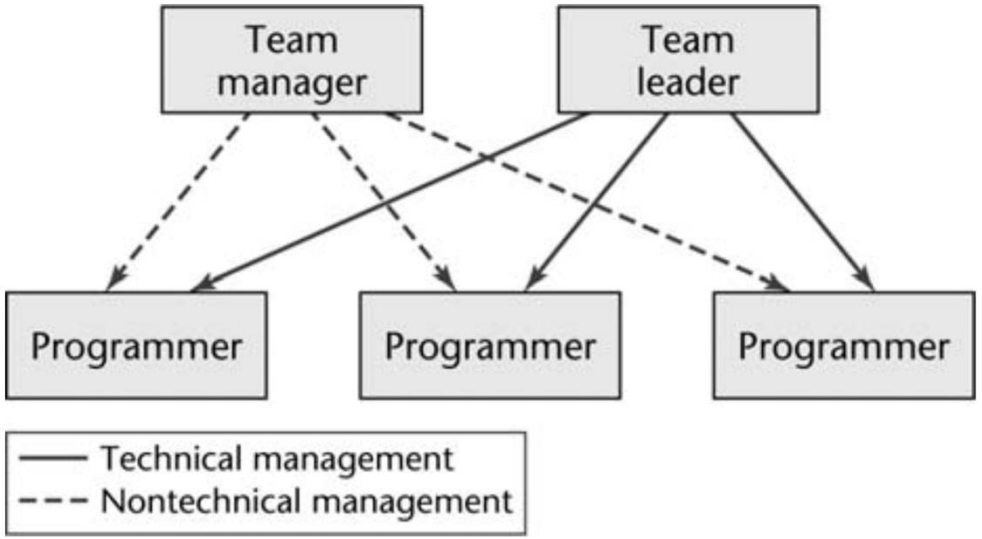
- Programmers
  - Do nothing but program
  - All other aspects are handled by the programming secretary
- Impracticalities of Approach
  - Chief programmers are difficult to find: Skilled programmer and successful manager
  - Back-up programmers very rare: Highly skilled, but just a backup
  - Programming secretary rare: Too much paperwork and software professionals' aversion to do paperwork

Team Manager & Team Leader Approach

- Beyond Chief Programmer and Democratic teams
- Chief programmer can be replaced by two individuals

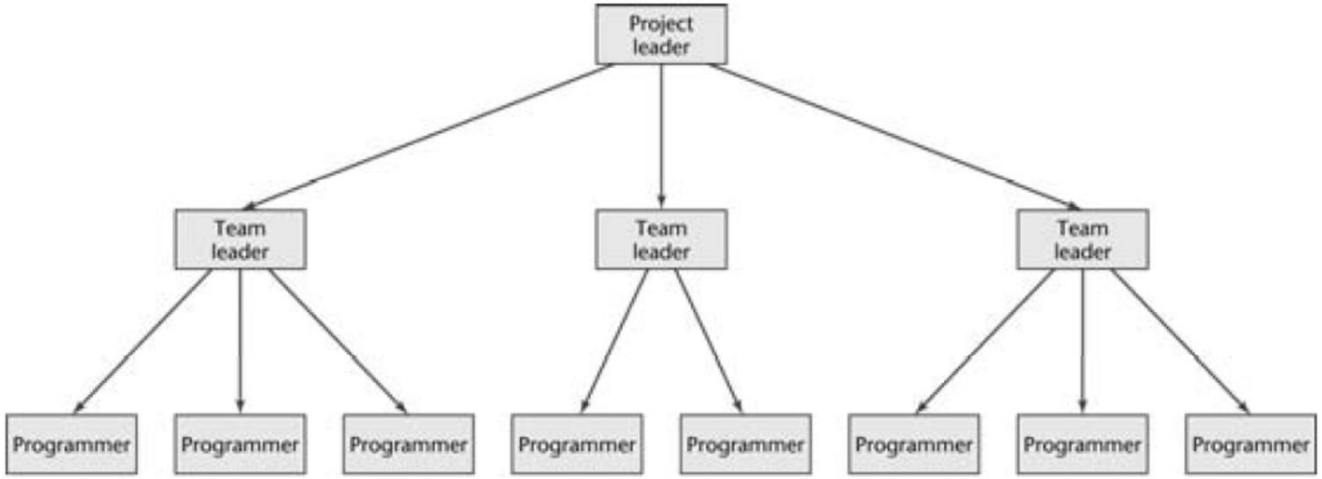
– Team leader — In charge of the technical aspects

– Team manager — Responsible for nontechnical managerial decisions



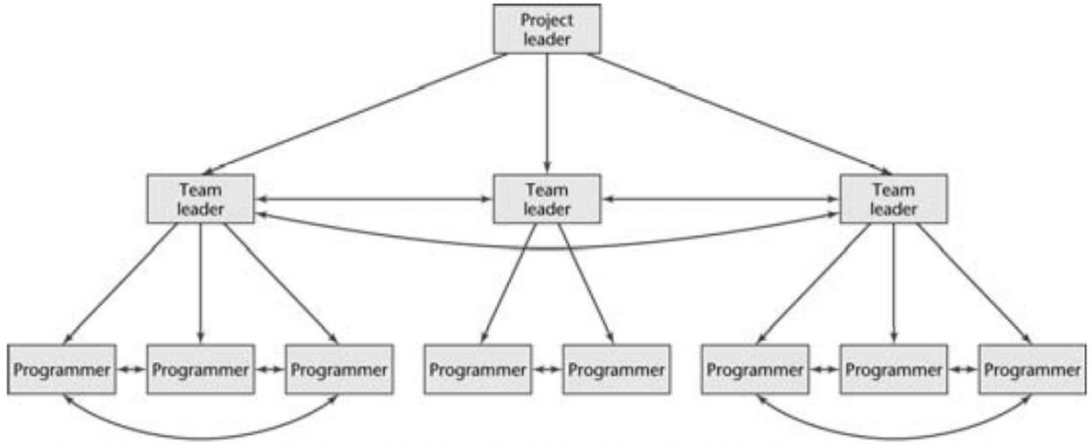
Team Manager & Team Leader Approach

- Scaling up for large projects
- Implementation of product under direction of project leader
- Programmers report to their team leaders
- Team leaders report to project leader
- For even larger products, additional levels can be added to hierarchy



Team Manager & Team Leader Approach

- Decentralizing decision-making process
- Combining best features of democratic and chief programmer teams
- Appropriate whenever a problem requires synergistic effect of group interaction
- New communication channels



Agile Team Roles

- Now we are going to look at team roles in two Agile development life cycles:
  - Scrum
  - Disciplined Agile Delivery (DAD)

Scrum Team Roles

- Scrum is an Agile framework for developing, delivering, and sustaining complex products, with an initial emphasis on software development.
- It has been used in other fields including research, sales, marketing and advanced technologies.
- It is designed for teams of ten or fewer members, who break their work into goals that can be completed within time-boxed iterations, called sprints: no longer than one month, most commonly two weeks.
- The Scrum Team tracks progress in 15-minute time-boxed daily meetings, called daily scrums.
- At the end of the sprint, the team holds sprint review, to demonstrate the work done, and sprint retrospective to continuously improve.

Scrum Team Roles

- Product owner

– Represents the product's stakeholders and the voice of the customer

– Bridges the communication gap between the team and its stakeholders, serving as a proxy for stakeholders to the team and as a team representative to the overall stakeholder community.

- Development team

– The development team has from three to nine members who carry out all tasks required to build increments of valuable output every sprint.

Scrum Team Roles

- Scrum master

– Accountable for removing impediments to the ability of the team to deliver the product goals and deliverables.

– Not a traditional team lead or project manager but acts as a buffer between the team and any distracting influences.

– Ensures that the Scrum framework is followed.

Disciplined Agile Delivery (DAD) Team Roles

- Disciplined Agile delivery (DAD) is the software development portion of the disciplined Agile toolkit.
- DAD enables teams to make simplified process decisions around incremental and iterative solution delivery.
- DAD builds on the many practices espoused by advocates of Agile software development, including Scrum, Agile modeling, lean software development, and others.
- In particular, DAD has been identified as a means of moving beyond Scrum.

Disciplined Agile Delivery (DAD) Team Roles

Primary roles

- Stakeholder

– More than just an end-user or customer, this is someone potentially affected by the development and deployment of a software project (e.g., accounting clerk)

- Product owner

– The person on the team who speaks as the "one voice of the customer", representing the needs of the stakeholder community to the Agile delivery team.

- Team member

– The team member focuses on producing the actual solution for stakeholders, including but not limited to: testing, analysis, architecture, design, programming, planning, and estimation.

- Team lead

– The team lead is a host leader and also the Agile coach, responsible for facilitating communication, empowering them to choose their way of working, and ensuring the team has the resources it needs and is free of obstacles.

- Architecture owner

– Owns the architecture decisions for the team and facilitates the creation and evolution of the overall solution design.

Disciplined Agile Delivery (DAD) Team Roles

Potential Supporting Roles

- Specialist/Technical Expert

– Although most Agile team members are generalizing specialists, sometimes other specialists are required depending on the needs of the project.

– These could be build masters, Agile database administrators, user experience (UX) designers, or security experts.

- Domain expert

– While the product owner represents a wide range of stakeholders, a domain expert is sometimes required for complex domains where a more nuanced understanding is required (e.g. finance expert).

- Independent tester

– Although the majority of testing is done by the DAD team members, in cases with complex domains or technology an independent testing team can be brought in to work in parallel to validate the work.

- Integrator

– For complex technical solutions at scale, an integrator (or multiple integrators) can be used to build the entire system from its various subsystems.

Scrum v.s. DAD

- **Choose Scrum if:** You have a small, single team needing a clear, simple starting point for Agile and high-frequency, short-cycle feedback.

- **Choose DAD if:** You need to tailor your process to a complex, regulated, or large enterprise environment, or if you need to integrate multiple techniques (e.g., Lean, Scrum, Kanban) across the entire delivery lifecycle.

Key Dynamics of Successful Teams

1. Psychological safety.

- Team members feel safe to take risks and be vulnerable in front of each other.
- They believe that their team members won’t punish or embarrass others for making and owning up to a mistake, suggesting a new idea, or asking a question.
- This is the most important dynamic and acts as the foundation for the other four.

2. Dependability.

- The team members get things done on time, and meet the team’s high bar for excellence.

Key Dynamics of Successful Teams

3. Structure and clarity.

- Team members have clear roles, plans & goals.

4. Meaning.

- The work/training is personally important to team members.
- The meaning is individual and personal to each team member.

5. Impact.

- The team members believe their effort matters and creates change.
- The impact is a subjective judgment that the work you are doing makes a difference and is contributing to the organization's goals.

Team Conflict

Definition

> Conflict happens when two or more people disagree and one of them (at least) decides to make an issue of it.

Conflict Examples in the Concept Phase

- First up, the Concept or Initiation Phase.
- This is at the start of the project while you are working out exactly what is going to happen.
- There's a risk of conflict between:

– The sponsor and users:

  About requirements or cost

  Around agreeing the problem and the solution

– The sponsor and the project manager

  Around the requirements and how to get there

– The sponsor and portfolio management

  About business case approval

Conflict Examples in the Definition Phase

- In this phase the project is being fully scoped and planned.
- There's a chance that you'll see conflict between:

– The project manager and team leaders
  - Due to a difference of opinion about planning
  - Around roles and responsibilities
  - Around assigning risk ownership to people who don't believe they are accountable for it

– The project manager and sponsor
  - Because once the detailed planning is done there may be a need to review scope.

– The project manager and suppliers
  - Around commercial agreements.

Conflict Examples in the Handover and Close Phase

- You've made it to the end of the project and now you're handing over to business as usual teams and closing the project down.
- What could go wrong here?
- You're at risk of project team conflict between:

– The project manager and the users or sponsor

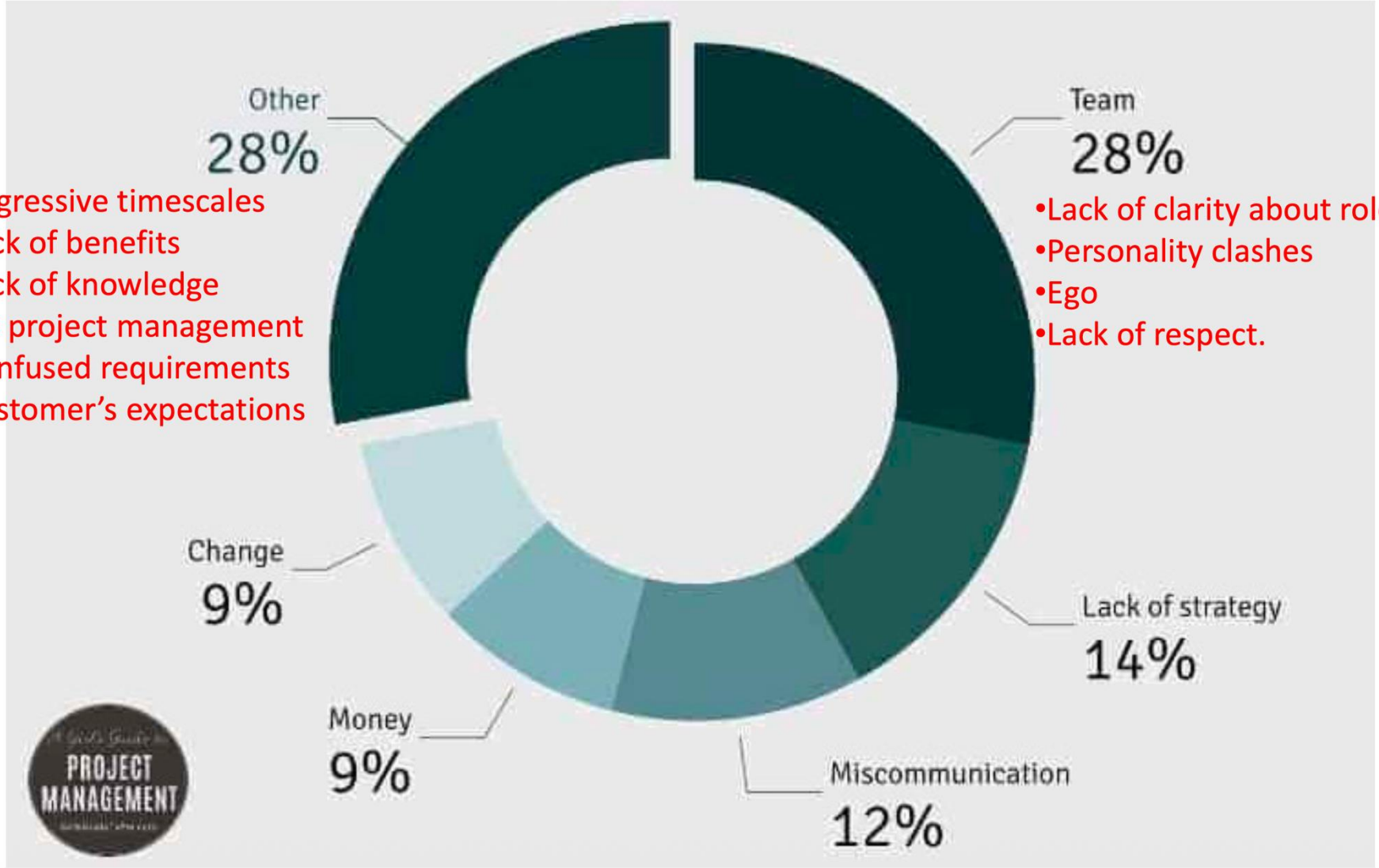
  - At the point of handover of deliverables

– The project manager and the operational team

  - As they might not want to receive the handover or take responsibility.

- Team conflict management should be on your agenda at this point in the project so you can tidy up any lingering awkward situations before you walk away from a project.

What Causes Conflict on Project Teams



- Aggressive timescales
- Lack of benefits
- Lack of knowledge
- No project management
- Confused requirements
- Customer's expectations

- Lack of clarity about roles
- Personality clashes
- Ego
- Lack of respect.

Potential Benefits of Team Conflict

Earlier Problem Identification

- Even the most seemingly trivial disagreements might stem from underlying unaddressed issues that, if not addressed, are likely to fester and then explode down the road.
- Similarly, conflict can identify practices and processes that need to be improved or replaced.

Better Problem-Solving

- The best ideas and solutions flow from healthy discussions involving a diversity of perspectives.
- It is challenging when our work colleagues disagree with our opinions or suggestions.
- If team members can learn to engage with these kinds of conflicts in constructive ways, then disagreements are not only normalized but can be seen to be an important piece of joint problem-solving.

Potential Benefits of Team Conflict

Healthy Relationships, Morale and Commitment

- Conflict that is denied, avoided, suppressed or handled ineffectively can harm relationships.
- Human beings can form inaccurate assumptions about the intentions of others which, unless surfaced and examined, can undermine important working relationships.
- On the other hand, if team members feel comfortable raising differing views, concerns or complaints and they see that these are heard and respected by their peers and management then their relationships with each other and with the organization can be strengthened.

Improved Productivity

- There will likely be an investment of time and energy at the outset to prepare individuals and teams to recognize and engage well with various kinds of workplace conflict.
- However, conflict that is handled well will free up people to focus on their jobs rather than tensions in the office which will lead to higher productivity, efficiency and effectiveness.

Potential Benefits of Team Conflict

Personal Growth and Insight

- Conflict situations can help us to learn more about ourselves and others.
- There is nothing like a difficult disagreement to reveal not only what we care about, but also our default approaches and reactions.
- We may not always show up as our “best selves” when in the midst of a heated discussion or when confronted by stinging criticism.
- However, in each of these situations, if we are open to it, then there is likely to be an important insight about ourselves that is worthy of learning.
- Self-awareness is the first step to managing ourselves better in the future.

Thomas-Kilmann Conflict Mode Instrument

- The Thomas-Kilmann conflict mode instrument (TKI) is a way of working out your preferred style for handling conflict:
  - Avoiding
  - Accommodating
  - Competing
  - Collaborating
  - Compromising
- There is no right or wrong style.
- Each style can be effective depending on the conflict and the individuals involve.
- Everyone has a preferred style.
- So understanding the styles of the individuals involved helps you choose an appropriate style.
- If one style isn’t working, try another one.

Avoiding

- Avoiding is where you don’t engage in conflict at all.
- It’s an unassertive approach, because you aren’t discussing or actively doing anything, and uncooperative, because you aren’t helping the other person either.
- It’s the ‘do nothing’ approach.
- It might sound like it would never work but it can be effective if you use it in moderation and in the right situations.

Example:

- Two team members are disagreeing about an issue in a team meeting and neither is changing their position.
- Tell them maybe we should set that issue aside to resolve later and move on.

Accommodating

- Accommodating is the opposite of Competing (more on that in a minute).
- This is what happens if you are directed to do something.
- For example, if your project sponsor directs you to take a certain course of action to handling a risk.
- It's an unassertive response but it shows high cooperation because your personal preferences are overlooked and overridden.
- You might think that if you end up in this situation, you've 'lost' but sometimes it isn't worth winning.

Example:

- A project team member checks their annual leave requests with you and you notice that a key task they are scheduled to do clashes with their time off.
- You're unhappy about them taking the time off, and they explain that they have already agreed a colleague will take over the work.
- It's not worth arguing this point as the work will still get done — so you agree to their leave request.
- Be careful how often you use this strategy because if you capitulate too often, people might start to see you as a walkover.

Competing

- Competing is also called Forcing.
- This is where you force the other person to take on your solution. They ‘lose’.
- You can only really take this approach when you have some kind of legitimate power that specifically relates to the situation you’re in. Examples of legitimate power include:
  - You are more senior than the other person in the management hierarchy
  - You have control of something relevant to the situation, like the budget or resources
  - You have expertise relevant to the situation that the other person doesn’t have
- Forcing is really only something to use when you really have to.

Example:

- You’re working on a building site and a colleague says they don’t want to wear the health and safety equipment.
- In this situation, you have to force the team member to comply with the regulations.

Collaborating

- This is an assertive way of resolving problems.
- It’s also very cooperative.
- Neither party tries to avoid the conflict or ‘win’.
- Instead, you work together to look into the problems and come up with solutions where you both get what you want.
- This is often seen as the ‘best’ way to manage conflict, but the ‘best’ way can be any of these.
- If a young child runs towards a busy road, collaborating isn’t the approach you’re going to use!
- All of the conflict resolution strategies can be useful to you in the right situation.

Compromising

- Finally, compromising.
- As you'd expect, both sides make concessions so that you can reach an agreement that unlocks the conflict.
- It's different from collaborating because neither party gets exactly what they were hoping for, but at least you reach a position where you can move on.

Key Takeaways

- Conflict management is part of the daily engagement with stakeholders and the team.
- Conflict can happen at any point in the project lifecycle.
- It can happen for numerous reasons, often to do with misunderstandings and misaligned expectations.
- There are many ways to resolve conflict at work.
- I gave you one technique (TKI)
- Deal with conflict as soon as you can to stop it
- becoming a larger problem!

Things to Remember

Psychological safety

- Team members feel safe to take risks and be vulnerable in front of each other.
- They believe that their team members won’t punish or embarrass others for making and owning up to a mistake, suggesting a new idea, or asking a question.

Dependability

- The team members get things done on time, and meet the team’s high bar for excellence.

Structure and clarity

- Team members have clear roles, plans & goals.

Meaning & Impact

- The work is personally important to team members.
- The team members believe their effort matters.
- Individual and personal to each team member.
- But, you can help other team members feel their work is meaningful and impactful by how you act.

Things to Remember

Establishment of goals and workplan of the team.

- Demonstrates leadership in the development of the goals and workplan, keeping the team focused throughout the planning.

Development of a collaborative team environment.

- Collaborates actively in the work of the team and often seeks out others for advice on strategies to meet team goals.

Inclusive team environment.

- Actively engages others for their opinion and participation in team discussions acknowledging the importance of their contribution.

Achievement of the team's goals.

- Completes own assigned tasks in a timely manner, while also assisting other team members to meet the overall goals.

Questions